

AWS 3-Tier Architecture

Detailed Project Implementation & Validation Document

Project: Building Scalable, Secure, and High-Performance Web Applications with AWS 3-Tier Architecture

Region: Asia Pacific (Mumbai) — ap-south-1

Repository: <https://github.com/Vennilavanguvi/Three-Tier-Architecture.git>

1. Project Objective

The objective of this project was to build and validate a three-tier web application architecture on AWS. Traffic is separated into a Web Tier, Application Tier, and Database Tier. Load balancers provide controlled traffic distribution, Nginx acts as the Web Tier reverse proxy, Flask provides the application service, and Amazon RDS MySQL provides the database layer.

Final traffic flow:

Client → Public ALB → Web EC2 / Nginx → Internal ALB → Private App EC2 / Flask → RDS MySQL

2. AWS Network Foundation

A custom VPC was created with CIDR 10.0.0.0/16. Six subnets were created across availability zones, with public subnets used for externally reachable/load-balancing components and private subnets used for application resources.

Resource	Configuration
VPC	10.0.0.0/16
Public Subnet AZ1	10.0.1.0/24
Public Subnet AZ2	10.0.2.0/24
Region	ap-south-1 (Mumbai)
Internet Gateway	Attached to custom VPC
NAT Gateway	Used for private subnet outbound access
Route Tables	Public and private routing configured

Validation: private application connectivity was tested with outbound access through NAT, confirming the private routing path was functioning.

3. Security Groups

Separate security groups were used to control communication between tiers. Web-SG protects the Web Server, App-SG protects the application tier, and Internal-ALB-SG controls access to the internal load balancer.

Important validated rule:

Internal ALB HTTP port 80 was permitted from Web-SG. The Web Server also required SSH/22 from the EC2 Instance Connect endpoint security group for administrative access.

4. Web Tier — EC2 and Nginx

A Web EC2 instance was deployed in a public subnet. Amazon Linux 2023 was used for the Web Server and Nginx was installed, enabled, and verified. The Nginx welcome page was initially used to confirm the web server was operational.

Nginx was then configured as a reverse proxy to the Internal ALB.

Nginx configuration used:

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://internal-Internal-ALB-1355428152.ap-south-1.elb.amazonaws.com;
        proxy_set_header Host $host;
```

```
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
}
```

The configuration was syntax-tested with **sudo nginx -t** and then reloaded. The default Nginx site configuration was removed to avoid a conflicting default server block.

5. Application Tier — Flask

The Application Tier was deployed in private subnets. The Flask application was prepared inside a Python virtual environment. Required packages included Flask, PyMySQL, and Gunicorn. The MySQL client was also installed for database testing.

Application setup sequence:

1. Update package information.
2. Install Python/pip/venv requirements.
3. Create and activate the virtual environment.
4. Install Flask, PyMySQL and Gunicorn.
5. Install MySQL client.
6. Create app.py.
7. Start the Flask service.
8. Test the application locally and through the load balancer.

The Flask service listens on port 5000 and returned the verified response **{"status":"Flask is running"}** during testing.

6. Database Tier — Amazon RDS MySQL

An Amazon RDS MySQL database instance was created as the managed Database Tier. A DB subnet group and security configuration were used so the application tier could communicate with the database without exposing the database as a public application endpoint.

The application database used the project database/table configuration established during implementation. Flask-to-RDS connectivity and successful data insertion were verified.

7. Internal Application Load Balancer

An internal Application Load Balancer named **Internal-ALB** was created inside the VPC. It uses HTTP port 80 and forwards requests to the application target group.

Internal ALB configuration:

Item	Value
Load Balancer	Internal-ALB
Scheme	Internal
Listener	HTTP : 80
Target Group	APP-TG
Application Port	5000
Security Group	Internal-ALB-SG

The Web Server successfully executed a curl request to the Internal ALB and received HTTP 200 with the Flask response. This verified the Internal ALB → Application Tier path.

8. Public Application Load Balancer

A public Application Load Balancer named **Public-ALB** was created across the two public subnets. The public listener forwards HTTP traffic to the Web Server target group **Web-TG** on port 80.

The Web Server was registered as a target in Web-TG and the public ALB was configured to forward client requests to it.

9. End-to-End Validation

The architecture was validated in stages instead of relying only on the final page.

Validation	Observed result
Web Server localhost	Nginx responded successfully
Web Server → Internal ALB	HTTP 200 and Flask response
Internal ALB → APP-TG	Application response received
Public ALB → Web-TG	Traffic reached Web Server
Public ALB end-to-end curl	Flask is running
Application → RDS	Database connectivity/data insertion verified

10. Final Architecture Flow

1. A client sends an HTTP request to the Public ALB.
2. Public ALB forwards the request to the Web Server target in Web-TG.
3. Nginx receives the request on the Web Server.
4. Nginx reverse-proxies the request to Internal-ALB.
5. Internal-ALB forwards the request to an Application Tier target in APP-TG.
6. Flask processes the request and communicates with RDS MySQL when required.
7. The application response travels back through the same logical path to the client.

11. Key Project Resources

Internal ALB DNS used during validation: internal-Internal-ALB-1355428152.ap-south-1.elb.amazonaws.com

Application target group: APP-TG

Web target group: Web-TG

Public load balancer: Public-ALB

Web security group: Web-SG

Internal ALB security group: Internal-ALB-SG

12. Submission Evidence Checklist

- VPC and subnet configuration
- Public/private route tables and NAT Gateway
- RDS MySQL status
- Application target group and healthy targets
- Internal ALB listener and target group
- Web EC2 / Nginx running
- Nginx reverse-proxy configuration

- Public ALB listener and Web-TG
- Final Public ALB curl response showing Flask is running

13. Conclusion

The project achieved the intended three-tier AWS architecture. The Web Tier is separated from the private Application Tier, the Application Tier communicates with the managed RDS MySQL Database Tier, and both public and internal load balancing paths were validated. The final end-to-end request successfully returned the Flask application response.

Prepared as the detailed implementation record for the completed AWS 3-Tier Architecture project.